



Prueba Técnica

Desarrollador Jr/Mid

Sistema de Gestión de Licitaciones

Tiempo estimado de entrega: 10 días

Contexto del Negocio

Una empresa necesita administrar licitaciones comerciales para sus clientes. Cada licitación pertenece a un cliente, contiene productos que la empresa planea vender, requiere un documento de propuesta formal, y debe enviarse oficialmente a la empresa cliente antes de que el plazo de presentación corra.

Tecnologías permitidas

Next.js, Python (Django/FastAPI), React, Vue, Angular, Laravel, Spring Boot, Node.js, Hono.js — full-stack o backend/frontend separado.

Integraciones obligatorias:

- Proveedor de email transaccional real (Resend, SendGrid, o similar — con plan gratuito), con soporte de adjuntos.
- Servicio de almacenamiento de archivos real (S3, Cloudinary, o Supabase Storage, UploadThing) para el documento de la propuesta.
- Mecanismo de tarea programada (cron, BullMQ + Redis, Celery beat, o cron externo) para el vencimiento automático y el recordatorio.

Base de datos: PostgreSQL corriendo en Docker o Supabase

Relaciones entre Entidades

- Un Cliente puede tener múltiples Licitaciones.
- Una Licitación pertenece a un solo Cliente.
- Una Licitación puede contener múltiples Productos (N:M, con cantidad y precio).
- Una Licitación tiene un documento de propuesta adjunto (referencia/URL del archivo subido).
- Una Licitación puede tener múltiples Pagos registrados (cuando está en estado por_cobrar).



- Cada cambio de estado de una Licitación genera un registro en el Historial de Transiciones (estado anterior, estado nuevo, usuario, fecha/hora).

Acceso y seguridad

- Login email/password, roles base (admin/user), JWT/Cookies.
- Auditoría (quién creó/modificó y cuándo) en cada entidad.
- Solo los admins pueden crear otros usuarios.

Estados de la Licitación y Tabla de Transiciones

Estados: ****activa****, ****finalizada****, ****por_cobrar****, ****cobrada****, ****perdida****.

Transiciones válidas (cualquier otra combinación debe rechazarse con error):

- borrador (implícito, antes de enviar) → activa: solo se permite si la licitación tiene documento de propuesta adjunto. Dispara el correo al cliente.
- activa → finalizada: la empresa ganó y completó la entrega.
- activa → perdida: manual (no se ganó la licitación) o automática (se venció la fecha límite sin resolución).
- finalizada → por_cobrar: se facturó, queda pendiente el cobro.
- por_cobrar → cobrada: automática, cuando el saldo pendiente llega a cero tras los pagos registrados.

Una licitación en finalizada, por_cobrar, cobrada o perdida no debe permitir agregar o quitar productos.

Reglas de Negocio Principal

1. El total de productos asociados a una licitación no debe superar su presupuesto máximo; si se excede, la API retorna error.
2. Toda licitación tiene una `fecha\_límite`. Si pasa la fecha límite y la licitación sigue en estado ****activa****, el sistema la transiciona automáticamente a ****perdida**** (job/chequeo programado).
3. Una licitación solo puede enviarse (pasar a ****activa****) si tiene un documento de propuesta adjunto subido al servicio de almacenamiento.
4. Al enviarse (transición a ****activa****), el sistema envía automáticamente un correo real a la empresa cliente con el resumen de la licitación (productos, presupuesto, fecha límite) y el documento adjunto.

5. Si la licitación sigue ****activa**** y faltan menos de 48 horas para la fecha límite, el sistema envía un correo de recordatorio a la empresa cliente (mismo mecanismo programado que el punto 2).
6. En estado ****por_cobrar**** se pueden registrar pagos contra la licitación. El saldo pendiente es el total facturado menos la suma de pagos; no se puede registrar un pago mayor al saldo pendiente. Al llegar el saldo a cero, la licitación pasa automáticamente a ****cobrada****.
7. Toda transición de estado debe quedar registrada en el historial (usuario, fecha, estado anterior/nuevo); solo se permiten las transiciones de la tabla anterior.

API mínimo esperado

- Clientes: creación y listado.
- Usuarios: creación y listado.
- Productos: creación y listado.
- Licitaciones: creación (borrador), listado, detalle (datos generales, productos, historial de estado), agregar/quitar productos (solo si editable), subir documento de propuesta, enviar (valida documento, dispara correo, transiciona a activa), marcar finalizada/perdida, registrar pago (solo si por_cobrar), historial de transiciones.

Requisitos Técnicos

- PostgreSQL Dockerizado o Supabase.
- Integración real de email con adjuntos (no simulada con console.log).
- Integración real de almacenamiento de archivos, con el documento accesible vía URL real.
- Job o chequeo programado funcionando en el ambiente desplegado (no solo en local) para vencimiento y recordatorio.

Entregables

- Repositorio Git público, README con instrucciones de instalación y ejecución.
- Despliegue en servicio gratuito (Vercel, Render, Netlify) con enlace funcional.
- Evidencia de un correo real recibido por el cliente con el documento adjunto (captura o log del proveedor).
- Evidencia de que el documento subido es accesible vía URL real.



Funcionalidades extra

- Swagger/OpenAPI si el API es independiente del frontend.
- Panel simple de licitaciones próximas a vencer (activas con fecha límite cercana).

Criterios de Evaluación

- Diseño del modelo de datos
- Claridad de arquitectura
- Organización del código
- Manejo de reglas de negocio
- Documentación,
- Despliegue funcional
- Manejo de lógica basada en tiempo (vencimiento automático + recordatorio).
- Integración real de email con adjuntos.
- Integración real de almacenamiento de archivos.

Resultado esperado

Una aplicación funcional donde una licitación se redacta, se le adjunta una propuesta, se envía formalmente al cliente con notificación automática, se le da seguimiento hasta su fecha límite con recordatorios, se resuelve (ganada o perdida) y, si se ganó, se factura y cobra hasta saldar, todo auditado paso a paso y desplegado para que un usuario final pueda probarlo de principio a fin.